

Шпаргалка по Nginx

Управление сервисом, устройство конфигурации, типовые блоки для статики, проксирования, HTTPS и балансировки, а также приёмы диагностики. Значения в угловых скобках замените своими.

Управление сервисом

```
nginx -t
    Проверить конфигурацию на ошибки, ничего не
    применяя. Делайте это перед каждой перезагрузкой

nginx -T
    Вывести итоговую конфигурацию целиком, со всеми
    подключёнными файлами

systemctl reload nginx
    Применить новую конфигурацию без разрыва текущих
    соединений

systemctl restart nginx
    Полный перезапуск: соединения обрываются

nginx -s quit
    Мягкая остановка: дождаться завершения запросов

nginx -v / nginx -V
    Версия; с заглавной — ещё и параметры сборки и
    список модулей
```

Где что лежит

```
/etc/nginx/nginx.conf
    Главный файл: глобальные настройки и блок http

/etc/nginx/conf.d/*.conf
    Подключаемые конфигурации сайтов

/etc/nginx/sites-available и sites-enabled
    Схема Debian/Ubuntu: файл создаётся в первом
    каталоге, символьная ссылка во втором включает сайт

/var/log/nginx/access.log, error.log
    Журналы запросов и ошибок по умолчанию
```

Структура конфигурации

Директивы живут в блоках (контекстах) и наследуются сверху вниз: main → http → server → location.

```
http {
    server {
        listen 80;
        server_name example.com;
        root /var/www/example;

        location / {
            try_files $uri $uri/ =404;
        }
    }
}
```

Отдача статики

```
root /var/www/site;
    Корень: путь запроса приписывается к нему целиком

alias /var/www/assets/;
    Подмена пути: часть URL из location отбрасывается

index index.html;
    Файл, который отдаётся при запросе каталога

try_files $uri $uri/ /index.html;
    Искать файл, затем каталог, иначе отдать указанный
    файл. Обычная схема для SPA

expires 30d;
    Заголовки кеширования для браузера

gzip on; gzip_types text/css application/json;
    Сжатие ответов указанных типов
```

Как выбирается location

```
location = /path
    Точное совпадение, наивысший приоритет

location ^~ /static/
    Префикс; при совпадении регулярные выражения не
    проверяются

location ~ /\.php$
    Регулярное выражение с учётом регистра (~* — без
    учёта)

location /
    Префикс по умолчанию: подходит, когда не подошло
    ничего точнее
```

Обратное проксирование

```
location /api/ {
    proxy_pass http://127.0.0.1:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

```
proxy_read_timeout 60s;
    Сколько ждать ответа от приложения

proxy_buffering off;
    Отдавать ответ потоком — нужно для SSE и длинных
    запросов

client_max_body_size 50m;
    Предел размера тела запроса; при загрузке файлов
    ошибка 413 лечится здесь

Косая черта в конце proxy_pass меняет поведение: с ней
префикс location отбрасывается, без неё путь передаётся
как есть.
```

WebSocket

```
location /ws/ {
    proxy_pass http://127.0.0.1:8080;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
```

HTTPS

```
server {
    listen 443 ssl;
    http2 on;
    server_name example.com;

    ssl_certificate
    /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key
    /etc/letsencrypt/live/example.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
}

server {
    listen 80;
    server_name example.com;
    return 301 https://$host$request_uri;
}
```

В версиях до 1.25 второй строкой писали `listen 443 ssl http2;`.

Балансировка нагрузки

```
upstream backend {
    least_conn;
    server 10.0.0.11:8080 weight=2;
    server 10.0.0.12:8080;
    server 10.0.0.13:8080 backup;
    keepalive 32;
}
```

`least_conn` / `ip_hash`

По числу активных соединений; по хешу адреса клиента — запросы одного клиента идут на один сервер

`max_fails=3` `fail_timeout=30s`

Через сколько неудач и на какое время сервер считается недоступным

Перенаправления и коды ответа

```
return 301 https://$host$request_uri;
    Постоянное перенаправление — самый дешёвый способ

rewrite ^/old/(.*)$ /new/$1 permanent;
    Перенаправление с преобразованием пути

return 444;
    Закрыть соединение без ответа — удобно против мусорных запросов

error_page 404 /404.html;
    Своя страница для кода ошибки
```

Ограничения и доступ

```
limit_req_zone $binary_remote_addr zone=lim:10m
rate=10r/s;
    Объявление зоны ограничения частоты запросов (в блоке http)

limit_req zone=lim burst=20 nodelay;
    Применение зоны в location с допуском всплесков

limit_conn_zone / limit_conn
    То же для числа одновременных соединений

allow 10.0.0.0/8; deny all;
    Доступ по адресам: правила проверяются по порядку

auth_basic "Закрытая зона"; auth_basic_user_file
/etc/nginx/.htpasswd;
    Простая парольная защита
```

Журналы и диагностика

```
access_log /var/log/nginx/site.log combined;
    Отдельный журнал для сайта; off отключает запись

log_format ext '$remote_addr $status $request_time';
    Свой формат: $request_time показывает медленные запросы

error_log /var/log/nginx/error.log warn;
    Уровни: debug, info, notice, warn, error, crit

tail -f /var/log/nginx/error.log
    Смотреть ошибки в реальном времени

curl -I -H "Host: example.com" localhost
    Проверить, какой server-блок отвечает, не трогая DNS

ss -ltnp | grep nginx
    На каких портах слушает сервер
```

Полезные переменные

- **\$host** — имя из заголовка Host или из `server_name`.
- **\$uri** — путь запроса после нормализации, без строки параметров.
- **\$request_uri** — исходный путь вместе с параметрами.
- **\$args** — строка параметров запроса.
- **\$remote_addr** — адрес клиента, **\$scheme** — http или https.
- **\$upstream_addr**, **\$upstream_status** — куда ушёл запрос и что ответило приложение.